

Universidade de Pernambuco
Professor: Emerson Remigio da Silva
Disciplina: Computação Gráfica
Discentes: Gabriel Gomes Barbosa,
Leonardo Marcelo S. Da Paz

Perguntas e Respostas para Apresentação — Jogo da Velha com Realidade Aumentada

1. Qual é a ideia principal do projeto?

A ideia principal do projeto é criar um **Jogo da Velha com Realidade Aumentada**, usando uma câmera para reconhecer QR Codes físicos e desenhar elementos virtuais sobre eles em tempo real.

O sistema reconhece três marcadores principais: *TABULEIRO*, *X* e *O*. O marcador *TABULEIRO* serve como referência para gerar a grade do jogo da velha, enquanto os marcadores *X* e *O* representam as peças dos jogadores. Essa proposta está alinhada com o objetivo descrito na Entrega 1, que é identificar QR Codes específicos, estimar suas coordenadas na imagem e renderizar sobreposições gráficas 2D sobre a cena real.

2. Onde está rodando a captura da câmera?

A captura da câmera está rodando dentro da classe *CameraThread*, que usa *cv2.VideoCapture(camera_id)* para abrir a webcam ou câmera virtual selecionada.

No código, a câmera é iniciada aqui:

```
camera_id = selecionar_camera()  
camera = CameraThread(camera_id)
```

```
camera.start()
```

```
471 def main():  
472     detector = cv2.QRCodeDetector()  
473     estado = EstadoJogo()  
474  
475     camera_id = selecionar_camera()  
476     camera = CameraThread(camera_id)  
477  
478     if not camera.is_opened():  
479         print("Erro: camera nao encontrada.")  
480         return  
481  
482     camera.start()  
483     criar_janela_video()  
484     tela_cheia = False
```

A função `camera.start()` cria uma thread separada que fica lendo os frames continuamente. Isso evita que a leitura da câmera trave o processamento principal da aplicação. No README, o projeto também descreve que a câmera fica lendo frames em segundo plano enquanto a thread principal executa detecção, geometria e renderização.

3. Por que foi usada uma thread separada para a câmera?

A thread separada foi usada para melhorar a fluidez da captura de vídeo.

Sem essa separação, o programa teria que capturar o frame, detectar QR Codes, calcular posições, desenhar overlays e exibir a imagem tudo na mesma sequência. Isso poderia causar travamentos ou atraso na imagem.

Com a classe `CameraThread`, a câmera fica atualizando o frame mais recente em segundo plano, enquanto a função principal usa sempre o frame mais atual disponível.

```
23 class CameraThread:
24     """Captura frames em uma thread separada para reduzir travamentos da leitura."""
25
26     def __init__(self, camera_id):
27         self.cap = cv2.VideoCapture(camera_id)
28         self.lock = threading.Lock()
29         self.frame = None
30         self.ret = False
31         self.running = False
32         self.thread = None
```

No código, isso acontece na função `_update()`:

```
def _update(self):
    while self.running:
        ret, frame = self.cap.read()

        with self.lock:
            self.ret = ret
            if ret:
                self.frame = frame

        time.sleep(0.001)
```

```
44
45 def _update(self):
46     while self.running:
47         ret, frame = self.cap.read()
48
49         with self.lock:
50             self.ret = ret
51             if ret:
52                 self.frame = frame
53
54         time.sleep(0.001)
55
56     def read(self):
57         with self.lock:
58             if self.frame is None:
59                 return False, None
60
61             return self.ret, self.frame.copy()
```

4. O Python trabalha com multithread real nesse projeto?

Em Python, existe uma limitação chamada **GIL**, que impede que várias threads executem código Python puro em paralelo de forma totalmente real. Porém, no projeto, a thread não foi usada para acelerar processamento pesado de CPU, mas sim para separar a **captura da câmera** do restante do fluxo.

Além disso, várias operações do OpenCV são implementadas em C/C++ por baixo, então o gargalo principal não é simplesmente “Python não ter multithread real”. O problema de instabilidade dos QR Codes está mais ligado à detecção visual, iluminação, foco, oclusão, tamanho dos QR Codes e quantidade de marcadores no mesmo frame.

5. Como o sistema detecta os QR Codes?

O sistema usa a classe *QRCodeDetector* do OpenCV:

```
detector = cv2.QRCodeDetector()
```

```
469     def main():
470         detector = cv2.QRCodeDetector()
471         estado = EstadoJogo()
472
473         camera_id = selecionar_camera()
474         camera = CameraThread(camera_id)
```

Dentro do laço principal, a detecção é feita com:

```
ok, decoded_info, points, _ = detector.detectAndDecodeMulti(frame)
```

```
495
496     ok, decoded_info, points, _ = detector.detectAndDecodeMulti(frame)
497
498     if ok:
499         processar_marcadores(estado, decoded_info, points, tempo)
500     else:
501         estado.expirar_candidato(tempo)
502
503     if estado.tabuleiro_pts is not None:
504         desenhar_tabuleiro(frame, estado.tabuleiro_pts, tempo)
505         desenhar_pecas_salvas(frame, estado, tempo)
506         desenhar_previa_candidato(frame, estado, tempo)
507         desenhar_resultado(frame, estado)
```

Essa função tenta detectar e decodificar múltiplos QR Codes no mesmo frame. Ela retorna o texto de cada QR Code e os quatro pontos da imagem correspondentes aos cantos do marcador.

O projeto foi planejado justamente para detectar as strings *TABULEIRO*, *X* e *O*, como descrito na proposta da Entrega 1.

6. O que significa *decoded_info*?

decoded_info é a lista com os textos encontrados dentro dos QR Codes.

Por exemplo, se a câmera detectar três QR Codes, o retorno pode ser algo como:

```
["TABULEIRO", "X", "O"]
```

Depois o código percorre essa lista junto com os pontos detectados:

```
for texto, pts in zip(decoded_info, points):  
    texto = texto.strip().upper()
```

Isso permite saber qual marcador foi detectado e onde ele está localizado na imagem.

7. O que significa *points*?

points contém as coordenadas dos quatro cantos de cada QR Code detectado. Esses pontos são essenciais porque permitem desenhar o elemento virtual sobre o marcador físico. Por exemplo, o *X* é desenhado usando as diagonais entre os cantos do QR Code, e o círculo é desenhado com base no centro e no tamanho do marcador. Na modelagem teórica do projeto, isso aparece como parte da etapa de “marcador detectado”, representado por texto mais quatro pontos 2D.

8. Como o sistema sabe se deve desenhar tabuleiro, *X* ou *O*?

Depois de detectar o texto do QR Code, o código compara o conteúdo com as opções esperadas:

```
if texto == "TABULEIRO":  
    desenhar_tabuleiro(frame, pts, tempo)
```

```
elif texto == "X":  
    desenhar_x(frame, pts, tempo)
```

```
elif texto == "O":
    desenhar_o(frame, pts, tempo)
```

```
441         if texto == "TABULEIRO":
442             estado.atualizar_tabuleiro(calcular_tabuleiro_virtual(pts))
443         elif texto in ("X", "O"):
444             pecas.append((texto, pts))
```

Ou seja, cada QR Code funciona como um marcador fiducial com uma função específica.

9. Onde o tabuleiro é desenhado?

O tabuleiro é desenhado na função:

```
def desenhar_tabuleiro(frame, pts, tempo):
```

```
293
294 def desenhar_tabuleiro(frame, tabuleiro_pts, tempo):
295     tabuleiro_int = tabuleiro_pts.astype(np.int32)
296     cv2.fillConvexPoly(frame, tabuleiro_int, BRANCO)
297
298     for i in range(1, 3):
299         t = i / 3
300         a = ponto_do_tabuleiro(tabuleiro_pts, t, 0).astype(int)
301         b = ponto_do_tabuleiro(tabuleiro_pts, t, 1).astype(int)
302         c = ponto_do_tabuleiro(tabuleiro_pts, 0, t).astype(int)
303         d = ponto_do_tabuleiro(tabuleiro_pts, 1, t).astype(int)
304         cv2.line(frame, tuple(a), tuple(b), VERDE, 5)
305         cv2.line(frame, tuple(c), tuple(d), VERDE, 5)
306
307     varredura = (tempo * 0.8) % 1
308     inicio = ponto_do_tabuleiro(tabuleiro_pts, 0, varredura).astype(int)
309     fim = ponto_do_tabuleiro(tabuleiro_pts, 1, varredura).astype(int)
310
311     cv2.polylines(frame, [tabuleiro_int], True, VERDE_ESCURO, 5)
312     cv2.line(frame, tuple(inicio), tuple(fim), AMARELO, 3)
313
```

Essa função recebe o frame da câmera, os pontos do QR Code *TABULEIRO* e o tempo atual.

Ela calcula o centro do marcador, a largura, a altura e expande uma área maior ao redor dele para desenhar a grade do jogo da velha.

O código calcula o centro assim:

```
centro = np.mean(pts, axis=0)
```

```
451
452     for simbolo, pts in pecas:
453         centro = np.mean(pts, axis=0)
454         celula = ponto_para_celula(estado.tabuleiro_pts, centro)
455
456         if celula is None:
457             continue
458
459         linha, coluna = celula
460         estado.registrar_candidato(simbolo, linha, coluna, tempo)
461
462         if estado.candidato == (simbolo, linha, coluna):
463             marcou_candidato = True
464
465     if not marcou_candidato:
466         estado.expirar_candidato(tempo)
```

Depois calcula um tamanho proporcional ao QR Code:

```
largura = np.linalg.norm(pts[0] - pts[1])
altura = np.linalg.norm(pts[1] - pts[2])
tamanho = max(largura, altura) * (3.4 + pulso * 0.15)
```

```
281     centro = np.mean(pts, axis=0).astype(int)
282     largura = np.linalg.norm(pts[0] - pts[1])
283     altura = np.linalg.norm(pts[1] - pts[2])
284     pulso = (np.sin(tempo * 5.0) + 1) / 2
285     raio = int(min(largura, altura) * (0.30 + pulso * 0.07) * max(progresso, 0.08))
286     espessura = int(7 + pulso * 4)
287     inicio = int((tempo * 140) % 360)
288     arco = int(300 * progresso)
```

```
195     def calcular_tabuleiro_virtual(pts_marcaador):
196         pts_marcaador = pts_marcaador.astype(np.float32)
197         centro = np.mean(pts_marcaador, axis=0)
198         largura = np.linalg.norm(pts_marcaador[0] - pts_marcaador[1])
199         altura = np.linalg.norm(pts_marcaador[1] - pts_marcaador[2])
200         tamanho = max(largura, altura) * TAMANHO_TABULEIRO
```

10. Por que foi usado NumPy no projeto?

O NumPy foi usado para facilitar os cálculos matemáticos envolvendo pontos, vetores, médias, distâncias e interpolação. Por exemplo, para calcular o centro de um marcador:

```
centro = np.mean(pts, axis=0)
```

Para calcular a distância entre dois pontos:

largura = np.linalg.norm(pts[0] - pts[1])

Esses cálculos são necessários para manter o desenho proporcional ao tamanho e à posição do QR Code. A Entrega 2 também descreve o uso do NumPy para normas vetoriais e coordenadas 2D dentro do módulo de geometria e mapeamento.

11. Como o X é desenhado?

O X é desenhado na função:

def desenhar_x(frame, pts, tempo):

```
258
259 def desenhar_x(frame, pts, tempo, mascarar=True, progresso=1.0):
260     pts = pts.astype(np.float32)
261     p1, p2, p3, p4 = pts
262
263     if mascarar:
264         mascarar_area(frame, pts)
265
266     pulso = int(7 + 3 * ((np.sin(tempo * 6.0) + 1) / 2))
267     progresso_final = progresso if progresso < 1.0 else 1.0
268
269     desenhar_linha_animada(frame, p1, p3, progresso_final, VERMELHO, pulso + 4)
270     desenhar_linha_animada(frame, p2, p4, progresso_final, VERMELHO, pulso + 4)
271     desenhar_linha_animada(frame, p1, p3, progresso_final, (40, 40, 255), pulso)
272     desenhar_linha_animada(frame, p2, p4, progresso_final, (40, 40, 255), pulso)
273
```

Primeiro, o código cobre o QR Code com uma área branca usando:

mascarar_area(frame, pts)

Depois desenha duas diagonais: uma do primeiro canto ao terceiro, e outra do segundo canto ao quarto.

desenhar_linha_animada(frame, p1, p3, progresso, VERMELHO, pulso + 4)

desenhar_linha_animada(frame, p2, p4, progresso, VERMELHO, pulso + 4)

Isso cria a sensação de que o X está sendo renderizado sobre o marcador físico.

12. Como o O é desenhado?

O O é desenhado na função:

def desenhar_o(frame, pts, tempo):

```
274
275 def desenhar_o(frame, pts, tempo, mascarar=True, progresso=1.0):
276     pts = pts.astype(np.float32)
277
278     if mascarar:
279         mascarar_area(frame, pts)
280
281     centro = np.mean(pts, axis=0).astype(int)
282     largura = np.linalg.norm(pts[0] - pts[1])
283     altura = np.linalg.norm(pts[1] - pts[2])
284     pulso = (np.sin(tempo * 5.0) + 1) / 2
285     raio = int(min(largura, altura) * (0.30 + pulso * 0.07) * max(progresso, 0.08))
286     espessura = int(7 + pulso * 4)
287     inicio = int((tempo * 140) % 360)
288     arco = int(300 * progresso)
289
290     cv2.ellipse(frame, tuple(centro), (raio, raio), 0, inicio, inicio + arco, AZUL, espessura + 4)
291     cv2.ellipse(frame, tuple(centro), (raio, raio), 0, inicio, inicio + arco, (255, 150, 50), espessura)
292
```

A função calcula o centro do QR Code:

centro = np.mean(pts, axis=0).astype(int)

Depois calcula o raio do círculo com base na largura e altura do marcador:

*raio = int(min(largura, altura) * (0.30 + pulso * 0.07))*

Por fim, usa *cv2.ellipse* para desenhar o círculo animado:

cv2.ellipse(frame, tuple(centro), (raio, raio), 0, inicio, inicio + 300, AZUL, espessura + 4)

13. Por que o QR Code é coberto por uma área branca antes do desenho?

O QR Code é coberto para melhorar o efeito visual de realidade aumentada. Em vez de deixar o QR Code aparecendo por baixo, o sistema desenha uma base branca sobre o marcador e depois desenha o X, o O ou o Tabuleiro por cima. Essa técnica melhora a sensação de que o elemento virtual substituiu o marcador físico.

```
8  BRANCO = (255, 255, 255)
9  VERDE  = (0, 220, 80)
```

14. Qual é a função de *mascarar_area()*?

A função *mascarar_area()* preenche a área do QR Code com uma cor sólida, normalmente branca.

```
190
191     def mascarar_area(frame, pts, cor=BRANCO):
192         cv2.fillConvexPoly(frame, pts.astype(np.int32), cor)
193
```

```
def mascarar_area(frame, pts, cor=BRANCO):
    cv2.fillConvexPoly(frame, pts.astype(np.int32), cor)
```

Ela usa *cv2.fillConvexPoly* para preencher o polígono formado pelos quatro pontos do QR Code. Na prática, essa função “apaga” visualmente o QR Code da imagem antes de desenhar o elemento virtual.

15. O que é realidade aumentada nesse projeto?

A realidade aumentada acontece porque o sistema combina a imagem real da câmera com elementos virtuais desenhados em cima dela. A câmera captura o ambiente real, o OpenCV identifica os QR Codes e o programa renderiza elementos gráficos como tabuleiro, X e O diretamente sobre o frame.

16. O projeto usa renderização 3D?

Não. A versão atual usa renderização 2D sobre o frame da câmera. O projeto ainda não faz estimativa completa de pose 3D da câmera nem renderiza objetos tridimensionais. Ele trabalha com primitivas gráficas 2D, como linhas, círculos, polígonos e textos.

17. Por que usar QR Code em vez de detectar X e O desenhados à mão?

O QR Code facilita a identificação porque ele carrega uma informação textual clara, como *TABULEIRO*, *X* ou *O*. Se o sistema tentasse reconhecer *X* e *O* desenhados à mão, seria necessário lidar com variações de formato, tamanho, inclinação, iluminação, espessura da caneta e ruído da imagem. Com QR Code, o *OpenCV* já retorna o texto decodificado e os pontos do marcador, tornando o projeto mais confiável para uma entrega acadêmica.

18. Qual é a limitação atual do projeto?

A principal limitação atual é a estabilidade da detecção de múltiplos QR Codes no mesmo frame. Quando há muitos marcadores, ou quando eles estão próximos, inclinados,

parcialmente cobertos ou em baixa iluminação, o OpenCV pode detectar alguns em um frame e perder outros no frame seguinte. Isso causa o efeito de “piscar”.

19. Como você pretende resolver o problema dos marcadores piscando?

A solução proposta é usar uma matriz interna de estado para salvar as jogadas. Em vez de exigir que a câmera veja todos os QR Codes o tempo inteiro, o sistema detecta apenas a jogada atual. Quando o jogador posiciona um QR Code *X* ou *O* sobre uma célula válida, o sistema confirma a jogada e salva essa informação na *matriz 3x3*.

Depois disso, o QR Code pode ser retirado, e o programa continua desenhando a peça com base no estado salvo.

Exemplo:

```
tabuleiro = [  
    ["X", "", ""],  
    [ "", "O", ""],  
    [ "", "", "X"]  
]
```

Essa abordagem separa a entrada física da lógica do jogo.

20. Por que essa solução da matriz interna é melhor do que detectar 9 QR Codes ao mesmo tempo?

Porque o jogo da velha não precisa rastrear todas as peças físicas o tempo inteiro. Depois que uma jogada é feita, basta salvar que aquela célula pertence ao jogador X ou O. A câmera só precisa identificar a próxima jogada. Isso reduz a quantidade de marcadores necessários no frame, melhora a estabilidade e deixa o sistema mais próximo de uma arquitetura real de jogo, com estado interno, turnos, validação de jogadas e verificação de vitória.

21. Como seria a lógica para confirmar uma jogada?

A lógica seria baseada em tempo e estabilidade. O jogador coloca o QR Code *X* ou *O* em cima de uma célula. O sistema identifica em qual célula o centro do marcador caiu. Se o marcador permanecer naquela célula por alguns segundos, por exemplo 2 ou 3 segundos, a jogada é confirmada. Isso evita que o sistema registre uma jogada acidental quando o QR Code passa rapidamente pela frente da câmera.

Exemplo de regra:

Se o QR Code X ficou sobre a célula [0, 1] por 2 segundos e essa célula está vazia,

então registra X nessa célula.

22. Como o sistema poderia saber em qual célula o QR Code foi colocado?

O sistema pode calcular o centro do QR Code da peça:

```
centro_peca = np.mean(pts_peca, axis=0)
```

Depois compara esse centro com a área do tabuleiro.

Como o tabuleiro é dividido em 3 linhas e 3 colunas, basta verificar em qual intervalo de coordenadas o centro da peça caiu.

Por exemplo:

Se o centro caiu no primeiro terço horizontal e segundo terço vertical, então a célula é linha 1, coluna 0.

23. Como será detectada a vitória no jogo?

A vitória pode ser detectada verificando as combinações possíveis na matriz 3x3.

Existem 8 possibilidades:

3 linhas

3 colunas

2 diagonais

Exemplo:

```
combinacoes = [  
    [(0,0), (0,1), (0,2)],  
    [(1,0), (1,1), (1,2)],  
    [(2,0), (2,1), (2,2)],  
    [(0,0), (1,0), (2,0)],  
    [(0,1), (1,1), (2,1)],  
    [(0,2), (1,2), (2,2)],  
    [(0,0), (1,1), (2,2)],  
    [(0,2), (1,1), (2,0)]  
]
```

Se três posições de uma dessas combinações tiverem o mesmo símbolo, o sistema declara vitória.

24. Como o projeto se relaciona com Computação Gráfica?

O projeto se relaciona com Computação Gráfica porque envolve renderização de elementos visuais sobre uma imagem real. Ele usa primitivas gráficas como linhas, círculos, polígonos e contornos. Também usa conceitos de transformação geométrica, escala, interpolação, rasterização e projeção 2D.

25. Como o projeto se relaciona com Visão Computacional?

O projeto usa Visão Computacional para capturar e interpretar a imagem da câmera. A parte de visão computacional está principalmente na detecção dos QR Codes, na obtenção dos pontos dos marcadores e no cálculo das posições na imagem. Depois que o sistema entende onde o marcador está, ele usa essa informação para desenhar elementos gráficos no lugar correto.

26. Qual é o pipeline geral do sistema?

O pipeline geral é:

Selecionar câmera
Capturar frame
Detectar QR Codes
Classificar marcador
Calcular geometria
Renderizar elementos virtuais
Exibir frame final

27. O que acontece dentro do *while True* da função *main()*?

O *while True* é o laço principal da aplicação.

Nele, o programa:

1. lê o frame mais recente da câmera;
2. detecta os QR Codes;
3. filtra os textos detectados;
4. organiza os marcadores;
5. desenha o tabuleiro, X ou O;
6. mostra o resultado na janela;
7. verifica se o usuário apertou ESC para sair.

Esse laço é o coração do programa, porque mantém a aplicação funcionando em tempo real.

28. Por que os marcadores são ordenados antes de desenhar?

O código faz isso:

```
marcadores.sort(key=lambda item: 0 if item[0] == "TABULEIRO" else 1)
```

Isso garante que o marcador *TABULEIRO* seja desenhado primeiro.

Essa ordem é importante porque o tabuleiro funciona como base visual do jogo. Depois dele, os símbolos *X* e *O* são desenhados por cima.

29. Como os QR Codes são gerados?

Os QR Codes são gerados no arquivo *gerar_qrcodes.py*.

O código cria três QR Codes:

```
codigos = ["TABULEIRO", "X", "O"]
```

Depois salva cada um como imagem PNG:

```
img.save(f"qr_{texto}.png")
```

Ou seja, o projeto gera automaticamente os marcadores necessários para a aplicação.

30. O que você ainda pretende melhorar no projeto?

As próximas melhorias são:

- implementar a matriz lógica 3x3;
- associar QR Codes X/O às células do tabuleiro;
- confirmar jogadas por tempo de estabilidade;
- impedir jogadas em células ocupadas;
- alternar turnos entre X e O;
- detectar vitória e empate;
- adicionar HUD com jogador atual e resultado;
- reduzir a tremulação dos marcadores.